

```
!pip install transformers torch --quiet
```

```
from transformers import pipeline, AutoTokenizer, AutoModel
import torch
```

```
from transformers import pipeline

mask_filler = pipeline("fill-mask")

input_sentence = "Hanoi is the <mask> of Vietnam."

predictions = mask_filler(input_sentence, top_k=5)

print("Câu gốc:", input_sentence)
for pred in predictions:
    print(f"Dự đoán: '{pred['token_str']}' với độ tin cậy: {pred['score']:.4f}")
    print(" ->", pred['sequence'])
```

No model was supplied, defaulted to distilbert/distilroberta-base and revision fb53ab8 (<https://huggingface.co/distilbert/distilroberta-base>) and revision fb53ab8 (<https://huggingface.co/distilbert/distilroberta-base>). Using a pipeline without specifying a model name and revision in production is not recommended.

Some weights of the model checkpoint at distilbert/distilroberta-base were not used when initializing RobertaForMaskedLM. This is expected if you are initializing RobertaForMaskedLM from the checkpoint of a model trained on another task or dataset, e.g. using `torch.load` to load a model checkpoint.

- This IS expected if you are initializing RobertaForMaskedLM from the checkpoint of a model trained on another task or dataset, e.g. using `torch.load` to load a model checkpoint.

- This IS NOT expected if you are initializing RobertaForMaskedLM from the checkpoint of a model that you expect to be initialized with those weights.

Device set to use cpu

Câu gốc: Hanoi is the <mask> of Vietnam.

Dự đoán: ' capital' với độ tin cậy: 0.9341

-> Hanoi is the capital of Vietnam.

Dự đoán: ' Republic' với độ tin cậy: 0.0300

-> Hanoi is the Republic of Vietnam.

Dự đoán: ' Capital' với độ tin cậy: 0.0105

-> Hanoi is the Capital of Vietnam.

Dự đoán: ' birthplace' với độ tin cậy: 0.0054

```
-> Hanoi is the birthplace of Vietnam.  
Dự đoán: ' heart' với độ tin cậy: 0.0014  
-> Hanoi is the heart of Vietnam.
```

```
# Câu hỏi 1: Mô hình dự đoán 'capital' hay không  
# Câu hỏi 2: Vì sao BERT phù hợp cho MLM?
```

```
answer_1 = ""  
1. CÓ! Mô hình dự đoán đúng từ 'capital' Và không chỉ đúng, mà còn đúng với độ tin cậy rất cao (0.9341).  
2. BERT phù hợp với Masked Language Modeling vì nó là mô hình Encoder-only,  
   có khả năng nhìn hai chiều (bidirectional), nên hiểu được ngữ cảnh cả trước và sau token bị mask.  
""  
print(answer_1)
```

```
1. Mô hình có dự đoán đúng từ 'capital' hay không hãy xem kết quả ở cell trước.  
2. BERT phù hợp với Masked Language Modeling vì nó là mô hình Encoder-only,  
   có khả năng nhìn hai chiều (bidirectional), nên hiểu được ngữ cảnh cả trước và sau token bị mask.
```

```
# 1. Load pipeline text-generation (GPT-2 mặc định)  
generator = pipeline("text-generation")  
  
# 2. Prompt ban đầu  
prompt = "The best thing about learning NLP is"  
  
# 3. Sinh văn bản tiếp theo  
generated_texts = generator(prompt, max_length=50, num_return_sequences=1)  
  
# 4. In kết quả  
print("Câu mời:", prompt)  
for text in generated_texts:  
    print("\nVăn bản được sinh ra:")  
    print(text["generated_text"])
```


No model was supplied, defaulted to openai-community/gpt2 and revision 607a30d (<https://huggingface.co/openai-c>)
Using a pipeline without specifying a model name and revision in production is not recommended.

config.json: 100%

665/665 [00:00<00:00, 66.1kB/s]

```
answer_2 = ""
```

1. CÓ - kết quả GPT-2 sinh ra hoàn toàn hợp lý theo hành vi của mô hình GPT-2.
2. GPT (Decoder-only) phù hợp với Next Token Prediction vì nó chỉ nhìn một chiều (unidirectional) - dự đoán token tiếp theo dựa trên các token trước đó.

```
""
```

```
print(answer_2)
```

tokenizer.json: 1.36M/? [00:00<00:00, 25.6MB/s]

1. Kết quả có hợp lý không? → xem output sinh văn bản từ GPT-2 ở cell trước.

Device set to use only 1 GPU
2. GPT (Decoder-only) phù hợp với Next Token Prediction vì nó chỉ nhìn một chiều (unidirectional) - dự đoán token tiếp theo dựa trên các token trước đó.
Truncation was not explicitly activated but `max\_length` is provided, please use `truncation=True` to ensure proper behavior. Setting `pad\_token\_id` to `eos\_token\_id`:50256 for open-end generation.

Both `max\_new\_tokens` (=256) and `max\_length` (=50) seem to have been set. `max\_new\_tokens` will take precedence over the latter one. The best thing about learning NLP is

```
# 1. Chọn mô hình
```

```
model_name = "bert-base-uncased"
```

```
tokenizer = AutoTokenizer.from_pretrained(model_name)
```

```
model = AutoModel.from_pretrained(model_name)
```

```
# 2. Input phrases
```

```
sentences = ["This is a sample sentence."]
```

```
# 3. Tokenization
```

```
inputs = tokenizer(sentences, padding=True, truncation=True, return_tensors='pt')
```

```
# 4. Lấy hidden states
```

```
with torch.no_grad():
```

```
    outputs = model(**inputs)
```

```
last_hidden_state = outputs.last_hidden_state # (batch, seq_len, hidden)
```

```
attention_mask = inputs["attention_mask"]
```

```
# 5. Mean pooling (bỏ padding)
```

```
mask_expanded = attention_mask.unsqueeze(-1).expand(last_hidden_state.size()).float()
```

```
sum_embeddings = torch.sum(last_hidden_state * mask_expanded, dim=1)
sum_mask = torch.clamp(mask_expanded.sum(1), min=1e-9)

sentence_embedding = sum_embeddings / sum_mask

# 6. In vector kết quả
print("Embedding của câu:")
print(sentence_embedding)
print("\nKích thước vector:", sentence_embedding.shape)
```



```
tokenizer_config.json: 100% 48.0/48.0 [00:00<00:00, 3.48kB/s]
config.json: 100% 570/570 [00:00<00:00, 44.1kB/s]
vocab.txt: 100% 232k/232k [00:00<00:00, 1.06MB/s]
tokenizer.json: 100% 466k/466k [00:00<00:00, 2.02MB/s]
model.safetensors: 100% 440M/440M [00:03<00:00, 197MB/s]
```

Embedding của câu:

```
tensor([[ -6.3874e-02, -4.2837e-01, -6.6779e-02, -3.8430e-01, -6.5784e-02,
          -2.1826e-01,  4.7636e-01,  4.8659e-01,  4.0647e-05, -7.4273e-02,
          -7.4740e-02, -4.7635e-01, -1.9773e-01,  2.4824e-01, -1.2162e-01,
           1.6678e-01,  2.1045e-01, -1.4576e-01,  1.2636e-01,  1.8635e-02,
           2.4640e-01,  5.7090e-01, -4.7014e-01,  1.3782e-01,  7.3650e-01,
          -3.3808e-01, -5.0331e-02, -1.6452e-01, -4.3517e-01, -1.2900e-01,
           1.6516e-01,  3.4004e-01, -1.4930e-01,  2.2422e-02, -1.0488e-01,
          -5.1916e-01,  3.2964e-01, -2.2162e-01, -3.4206e-01,  1.1993e-01,
          -7.0148e-01, -2.3126e-01,  1.1224e-01,  1.2550e-01, -2.5191e-01,
          -4.6374e-01, -2.7261e-02, -2.8415e-01, -9.9249e-02, -3.7017e-02,
          -8.9192e-01,  2.5005e-01,  1.5816e-01,  2.2701e-01, -2.8497e-01,
           4.5300e-01,  5.0945e-03, -7.9441e-01, -3.1008e-01, -1.7403e-01,
           4.3029e-01,  1.6816e-01,  1.0590e-01, -4.8987e-01,  3.1856e-01,
           3.2861e-01, -1.3403e-02,  1.8807e-01, -1.0905e+00,  2.1009e-01,
          -6.7579e-01, -5.7076e-01,  8.5947e-02,  1.9121e-01, -3.3818e-01,
           2.7744e-01, -4.0539e-01,  3.1305e-01, -4.1197e-01, -5.6820e-01,
          -3.9074e-01,  4.0747e-01,  9.9898e-02,  2.3719e-01,  1.0154e-01,
          -2.5670e-01, -2.0583e-01,  1.1762e-01, -5.1439e-01,  4.0979e-01,
           1.2149e-01,  1.9333e-02, -5.9029e-02, -2.0141e-01,  7.0860e-01,
          -6.4609e-02,  2.4779e-02, -9.0578e-03,  1.9666e-02,  3.0815e-01,
          -4.9832e-02, -1.0691e+00,  6.1072e-01, -4.9722e-02, -1.5156e-01,
          -6.7778e-02,  4.7812e-02,  5.2103e-01,  1.6951e-01,  1.0146e-02,
           5.3093e-01, -7.8189e-02,  6.5843e-02, -2.9382e-01, -4.6045e-01,
           4.2071e-01,  1.1822e-01,  2.3631e-01, -4.5379e-02, -1.3740e-01,
          -4.4018e-01, -6.8123e-02,  1.9935e-01,  8.7062e-01, -2.2603e-01,
           3.3604e-01,  2.0236e-01,  3.7898e-01,  1.9533e-01, -3.0366e-01,
           3.8633e-01,  6.1949e-01,  6.8663e-01, -1.8968e-01, -3.6815e-01,
          -1.6616e-01, -7.0827e-02, -3.4610e-01, -8.5326e-01,  4.6645e-02,
           2.8512e-01,  1.0890e-01,  2.5938e-01, -4.2975e-01,  4.3345e-01,
           2.0637e-01, -3.8656e-01, -3.8187e-02,  3.6925e-01,  3.0130e-01,
           4.0251e-01,  1.2887e-01, -3.7689e-01, -3.4447e-01, -4.2116e-01,
          -1.0252e-01, -8.9737e-02,  4.7384e-01,  8.1717e-02,  1.5885e-01,
```

```
7.6674e-01, 3.4493e-01, 9.8538e-04, 4.8932e-02, 2.6132e-01,
3.8329e-02, -2.0036e-01, 2.6654e-01, 9.3773e-02, -4.6779e-02,
-4.0519e-01, -4.4310e-01, 6.1268e-01, -1.8950e-01, -3.8333e-01,
2.0583e-01, 1.5379e-01, -1.4664e-01, 5.3847e-01, -3.9618e-01,
-2.0599e+00, 6.7052e-01, 2.1112e-01, -4.7306e-01, 3.4865e-01,
-2.9919e-01, 5.4614e-01, -5.3924e-01, -2.4877e-01, -2.9070e-02,
-2.0319e-01, -7.3275e-02, -3.8147e-01, -5.4454e-01, 3.5049e-01,
-1.1249e-01, -2.1471e-01, -3.8439e-01, -1.0760e-01, -8.8821e-02,
2.5263e-01, 2.1448e-01, 5.5799e-02, -6.5411e-02, 9.9837e-02,
3.3435e-01, 2.4018e-01, 2.9875e-02, -1.1191e-01, 5.4330e-01,
-5.5214e-01, 1.1125e+00, 5.4141e-01, -7.4160e-02, 3.5337e-01,
1.2313e-01, 3.4855e-02, -2.8568e-01, -1.2517e-01, -4.4332e-02,
1.3323e-01, -2.4995e-01, -4.9833e-01, 4.1959e-01, -3.1580e-01,
6.1942e-01, 3.1113e-01, 4.8846e-01, 6.1518e-01, -3.6326e-02,
2.1294e-02, -3.5715e-01, 5.9126e-01, 1.5102e-01, -2.9641e-01,
2.9441e-01, -1.4138e-01, 1.1662e-01, -3.6223e-01, -1.4621e-01,
6.5254e-02, 3.9270e-01, 3.8543e-01, -2.3996e-01, -3.1482e-01,
-4.6860e-01, -1.1920e-01, 8.6236e-02, -3.4596e-02, -3.6275e-01,
-3.9838e-01, -3.6006e-01, -1.9672e-01, -2.7738e-01, -4.1097e-01,
3.6456e-01, -2.6012e-01, 1.2587e-01, 1.2752e-01, 5.4261e-01,
1.0569e-01, 3.5704e-01, 1.4766e-01, 4.4929e-01, -8.1255e-01,
-3.0409e-02, 5.8063e-02, 2.0699e-01, 6.6129e-01, 3.9243e-01,
-6.8644e-01, -8.3415e-01, -1.2653e-01, 1.9644e-01, -4.0900e-01,
-6.3777e-02, -1.8780e-01, 7.9473e-02, -1.7443e-01, 3.1936e-01,
3.6761e-01, 4.3044e-01, -1.7471e-01, 1.3718e-01, 1.4272e-01,
-6.0642e-01, 2.3549e-01, 2.7794e-01, 1.0539e-01, -4.5836e-01,
-3.2561e-01, 1.5292e-02, -2.7672e-01, -4.8611e-01, 3.9087e-01,
3.6016e-01, 6.3403e-01, -1.2816e-01, -1.6720e-02, -3.0123e-01,
-1.7321e-01, -6.7296e-01, -2.7015e-01, -1.2534e-01, -8.0565e-01,
3.6115e-01, 1.7370e-01, -3.5578e-01, -2.1725e+00, -2.8102e-02,
-2.6773e-02, -2.2444e-01, 3.1249e-02, 6.4420e-02, -1.5017e-01,
-3.4460e-01, -5.5676e-01, 1.8039e-01, -4.2200e-01, -9.1074e-01,
-3.1339e-03, 7.2439e-01, 3.9006e-01, -4.4129e-02, -4.4785e-02,
2.8707e-02, -1.2432e-01, 6.9166e-01, -1.3227e-02, -2.3540e-02,
-7.0615e-02, -4.5062e-01, 4.5705e-01, 3.3198e-01, -2.2727e-01,
3.2434e-01, -4.5709e-01, -5.1586e-01, -1.5693e-01, -1.0897e-01,
3.9317e-01, -2.5950e-01, -1.5326e-01, 3.3276e-01, 3.2522e-01,
-2.5241e-01, 4.7946e-01, -3.7339e-01, -2.8146e-01, 7.7628e-02,
2.7131e-01, -3.7212e-01, 6.1400e-01, -2.9269e-01, -4.4389e-01,
-3.7750e-01, 2.7135e-01, 3.6869e-01, -1.6904e-01, -1.7583e-01,
2.9626e-01, 2.9393e-01, -8.2036e-03, 3.4545e-02, 4.5846e-01,
3.0137e-01, 1.6171e-01, -2.7772e-01, 5.2397e-01, -6.1950e-01,
-2.4818e-02, -5.1942e-02, 3.6764e-01, -5.8404e-01, -2.6651e-01.
```



```
-7.5761e-02, -1.7428e-01, 4.1535e-01, -2.7556e-01, -5.6796e-02,  
-4.3509e-01, -9.6659e-01, -1.1800e-01, -3.8004e-01, 2.7555e-01,  
-2.9743e-01, 2.4023e-01, -3.8869e-01, -4.0248e-01, -8.3882e-01,  
-1.0652e-01, -9.4192e-02, 1.4810e-01, 9.0844e-03, 1.4658e-01,  
-1.4813e-01, -1.6078e-01, -4.3130e-01, -8.0683e-02, 4.3722e-01,  
4.2623e-01, 3.3201e-01, -2.8283e-01, 2.0751e-01, 5.9093e-01,  
-6.3453e-01, 5.7386e-01, -2.9870e-01, 1.0221e-02, -4.7624e-01,  
4.9509e-01, 4.7470e-02, 1.3193e-01, 3.6281e-01, -1.1642e+00,  
3.8372e-01, 1.7071e-01, 3.8881e-01, 1.7703e-01, -4.7019e-01,  
1.2768e-01, -1.3409e-01, -2.8794e-01, 3.2066e-01, -3.7853e-01,  
4.6259e-01, 5.2343e-01, 3.0741e-01, 2.7410e-01, 4.9933e-01,  
-5.6466e-01, -3.4677e-01, -6.6571e-01, -1.3347e-01, -8.5910e-02,  
6.2487e-02, -3.9922e-01, -3.5880e-01, -5.8337e-01, -1.3556e-02,  
-1.6812e-01, 1.3949e-01, 2.9142e-01, -4.5623e-01, -1.0705e-01,  
6.6569e-01, 7.6614e-01, -1.9306e-01, 4.3854e-01, 2.8110e-01,  
-3.6835e-01, -1.6012e-01, -2.5005e-01, 7.6297e-01, 1.9653e-01,  
-1.8120e-01, 1.1895e-03, 1.8755e-01, -1.8990e-01, -2.3725e-01,  
3.2633e-02, -2.7723e-01, -4.7986e-02, -6.2332e-01, 2.6807e-01,  
-1.2293e-01, -2.7098e-01, -6.9677e-01, 1.5738e-01, 5.3557e-01,  
1.2760e-01, -1.7979e-02, 1.2769e-01, -5.6453e-02, 6.7965e-02,  
1.8555e-01, -3.6374e-01, 2.8518e-01, -4.3920e-01, -2.4276e-01,  
5.1755e-01, -2.3519e-01, 6.4010e-02, 3.9268e-01, 5.7986e-01,  
-1.7500e-01, 7.1669e-02, 5.7915e-01, 5.1699e-02, -1.1085e-03,  
-4.8444e-02, 1.5531e-01, 2.8402e-01, 6.8268e-01, 8.1524e-02,  
1.5325e-01, 1.9466e-01, 1.2260e-02, -3.3223e-01, 2.5763e-02,  
-1.6071e-01, -3.7663e-01, -7.3670e-01, -5.0067e-01, 1.1540e-01,  
-3.3788e-01, 1.2889e-01, 2.1528e-02, 6.1149e-01, 3.3550e-01,  
-2.0217e-01, -6.3961e-02, 2.4056e-02, -9.3070e-02, -2.7771e-02,  
1.8373e-01, -4.1812e-02, -1.0456e-01, -2.7569e-01, -3.9216e-01,  
-3.2092e-01, -1.0158e+00, 1.6407e-01, 4.5044e-02, 2.3079e-01,  
2.6936e-02, -2.1047e-01, -3.1392e-01, -4.6154e-01, -4.0347e-01,  
7.3271e-02, 1.1470e-01, -2.4129e-01, -3.6199e-01, -5.3254e-01,  
-5.2185e-01, -4.0713e-01, 2.1619e-02, 1.4186e-01, -1.2105e-01,  
-1.4055e-02, -4.2986e-02, -1.2459e-01, -6.6652e-01, -6.4169e-01,  
-2.2399e-01, 6.2557e-02, -3.3323e-01, 1.8865e-02, 1.6465e-01,  
-2.8729e-02, -5.9477e-01, 2.0963e-02, -3.3761e-01, 1.8088e-01,  
7.4363e-01, 1.5554e-01, 2.7824e-01, -2.1975e-01, 5.1316e-01,  
-3.9708e-01, -2.4769e-01, 4.3027e-01, -2.3078e-01, -2.9392e-01,  
1.3250e-01, -6.1646e-01, 2.6501e-01, 5.6891e-01, -1.3585e-01,  
-1.2774e-01, 8.1189e-01, 3.6497e-01, 5.0178e-01, 2.9736e-01,  
8.7772e-01, 7.3390e-02, 2.5788e-01, -3.3609e-01, 8.8207e-02,  
2.1282e-02, 1.4487e-01, 7.6676e-03, -3.9123e-01, -6.3919e-02,  
2.7226e-01, 8.2042e-02, 2.0821e-02, 2.4520e-02, 2.0262e-01
```

```
answer_3 = ""
```

1. Vector có kích thước (1, 768). Đây là hidden_size = 768 của BERT-base.

2. attention_mask được dùng trong Mean Pooling để không tính trung bình qua các token PAD.

```
""
```

```
print(answer_3)
```

```
4.3334e-01, -4.0269e-02, -5.8663e-01, 7.2658e-01, -5.5645e-01,
-5.7467e-02, 1.0528e-01, 1.0038e-01, 2.5418e-03, 7.7563e-01,
1.9355e-01, 6.4184e-01, 5.9658e-01, 2.1974e-02, 1.8323e-01,
1.7593e-01, 4.8541e-01, -4.6240e-01, 3.5692e-01, 3.2622e-01,
```